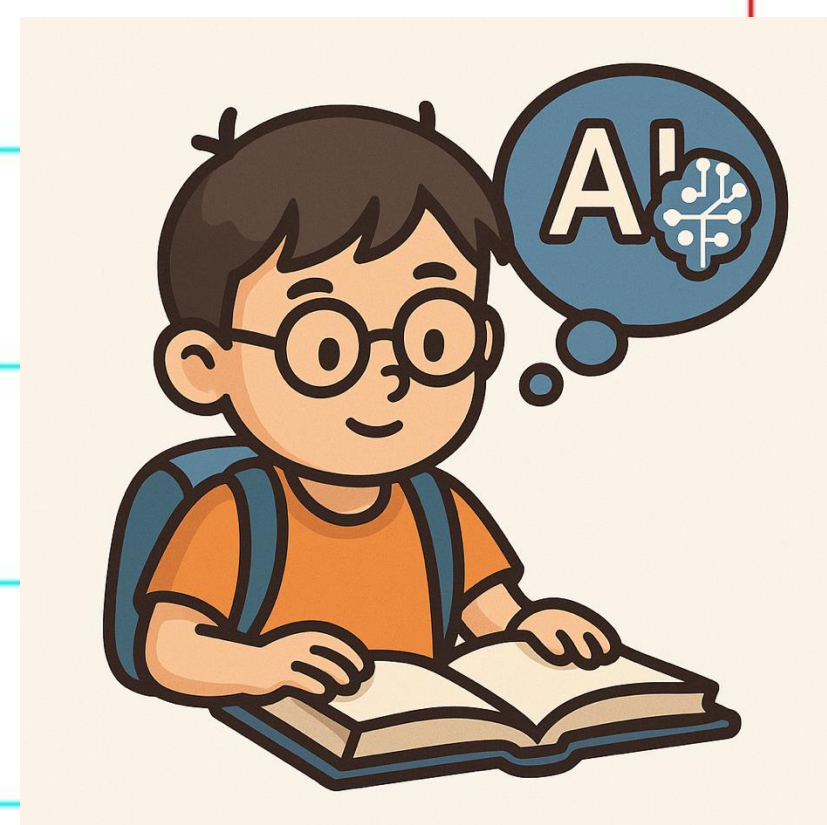


AI in the Browser: Practical ML with JavaScript



Ron Dagdag

Code > Create > Coach > Repeat.

R&D Engineering Manager at



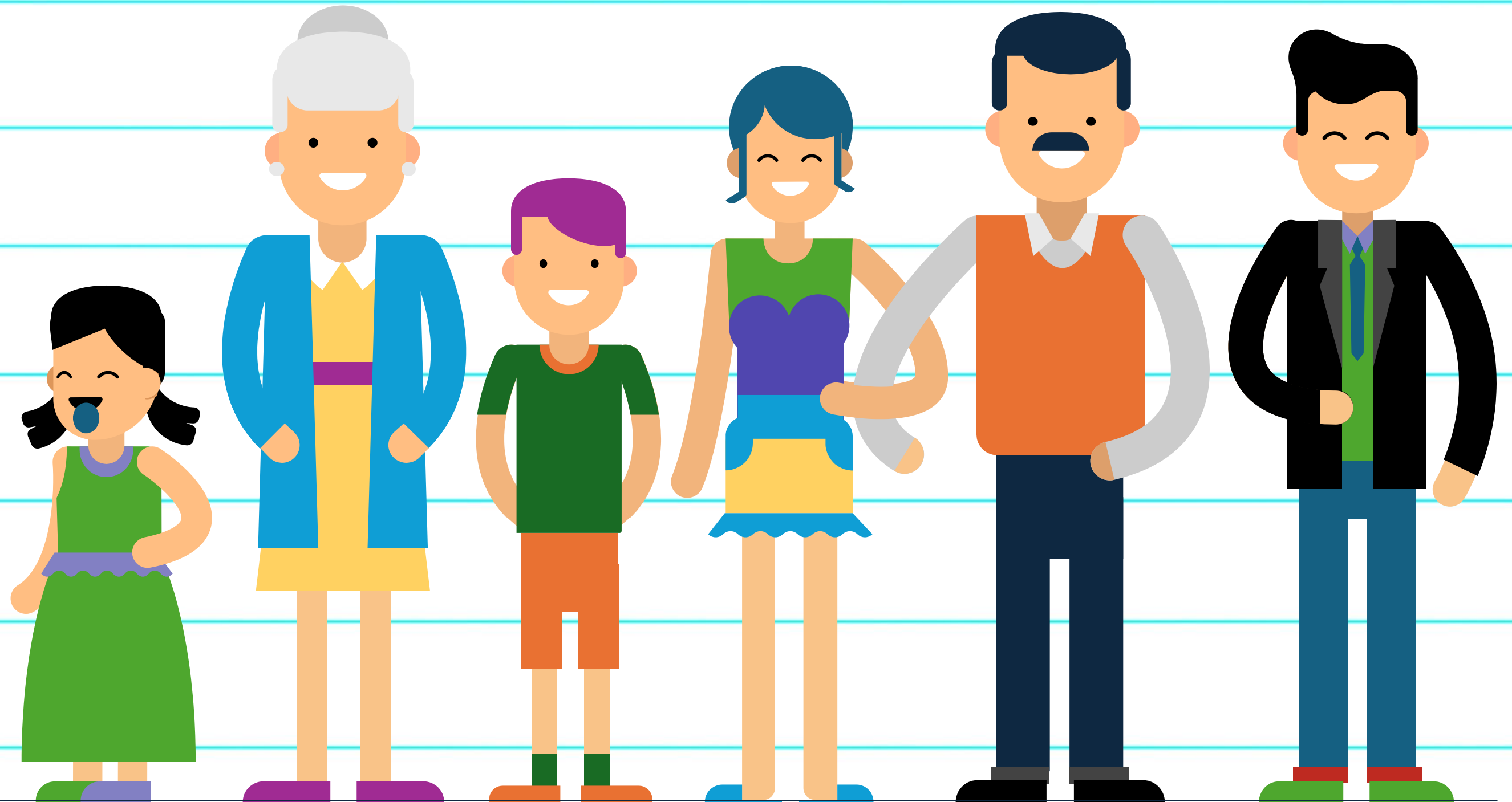
Microsoft®
Most Valuable
Professional



Award Categories

AI, Windows Development,
Internet of Things, Mixed Reality

Generational Marketing



Generational Marketing



The Greatest Generation

Built resilience through the Great Depression and defined heroism in WWII.

1901–
1924

The Silent Generation

Hardworking, disciplined, and shaped by post-war stability.

1925–
1945

The Baby Boomer Generation

Grew up in prosperity, fueled cultural revolutions, and reshaped work and retirement.

1946–
1964

Generation X

The independent, skeptical, latchkey kids who bridged analog and digital worlds.

1965–
1979

Millennials

Tech-savvy, purpose-driven, and shaped by 9/11, social media, and the gig economy.

1980–
1994

Generation Z

Digital natives, socially conscious, and redefining norms in work, activism, and identity.

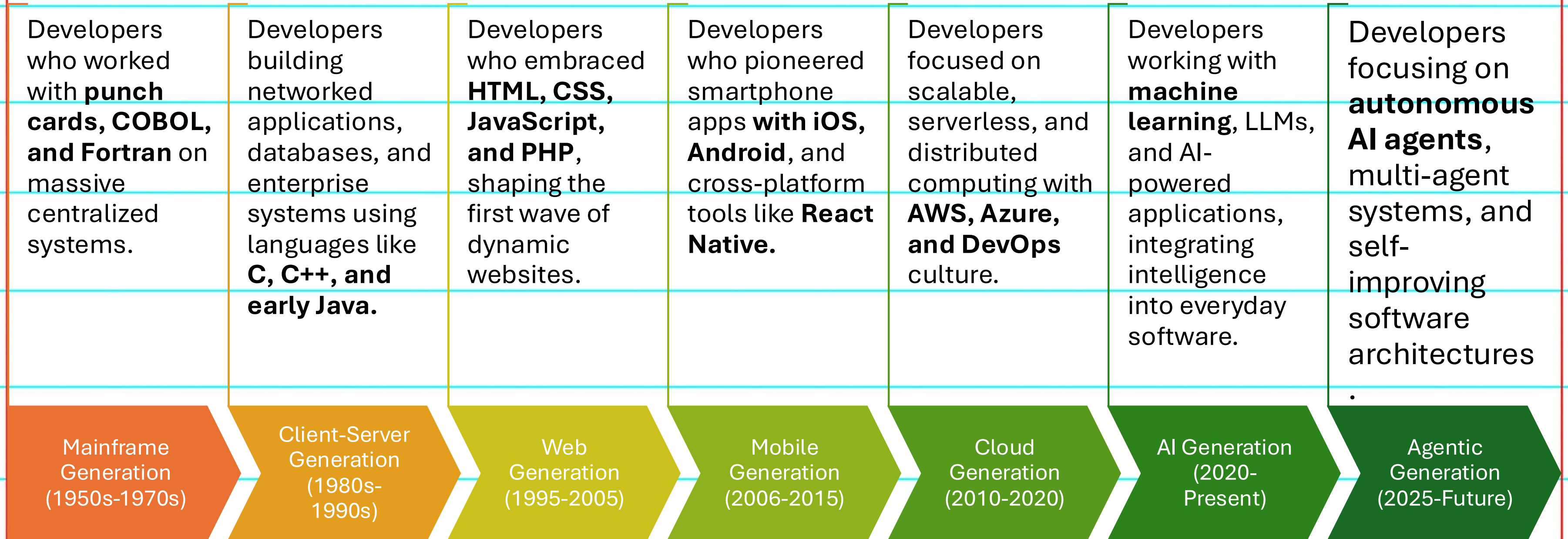
1995–
2012

Gen Alpha

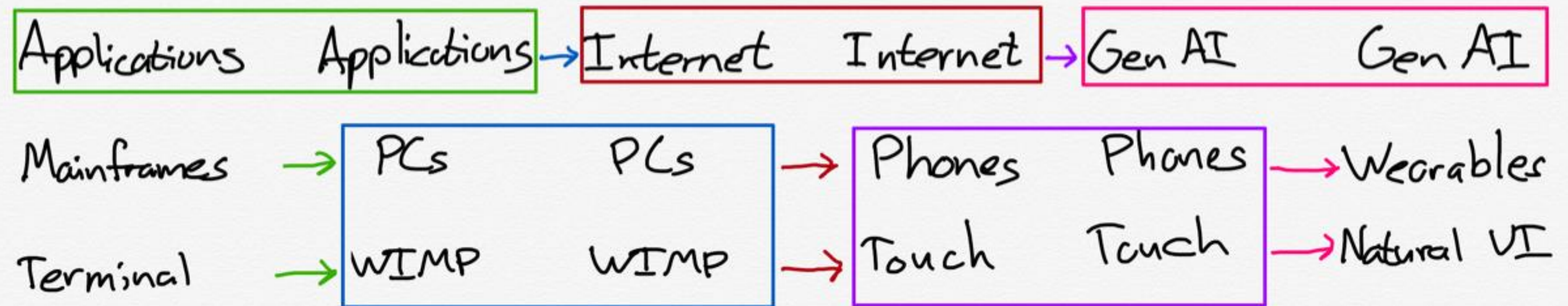
touchscreen-first generation growing up in an always-connected world.

2013–
2024

Developer Generations

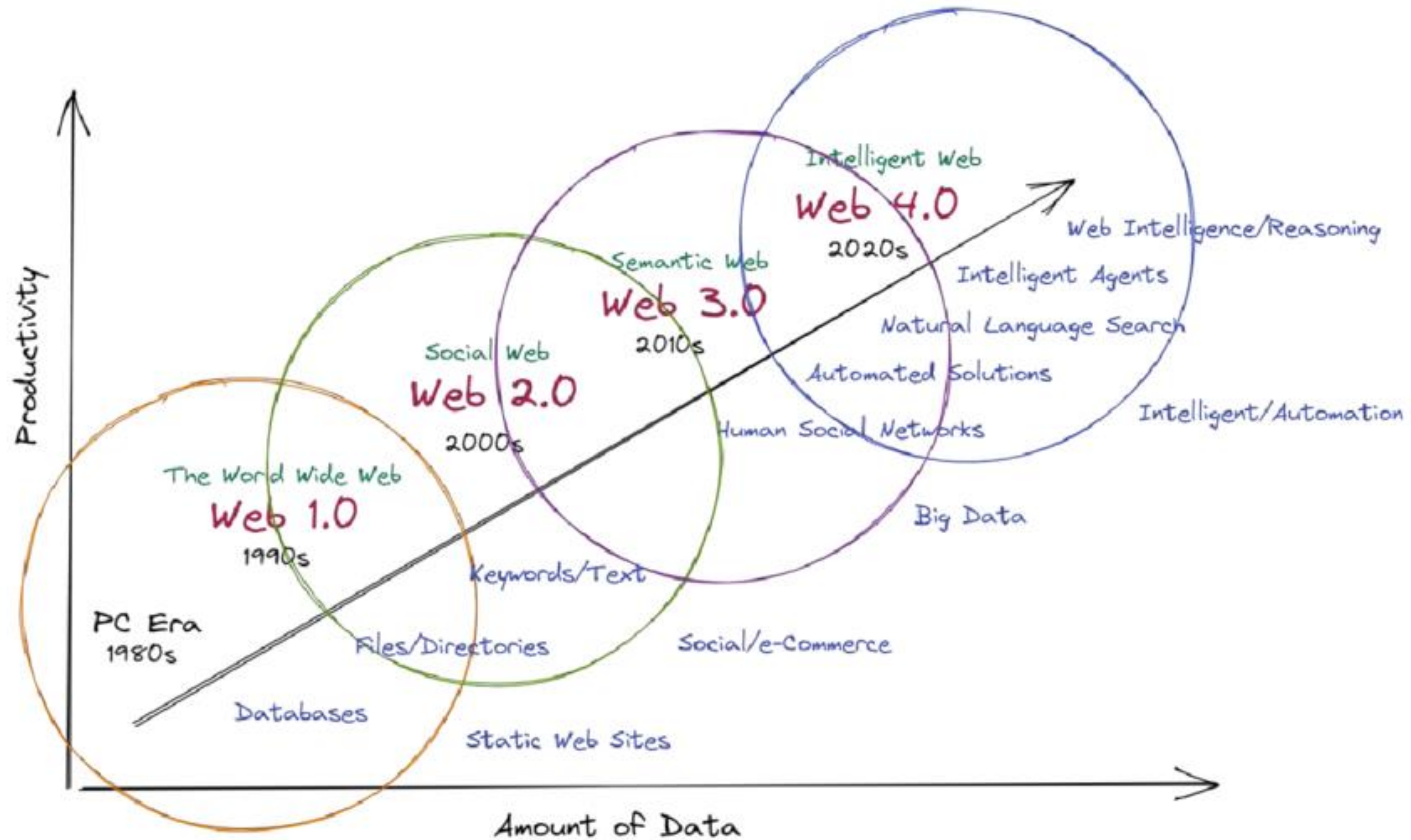


Technology Bridges



Stratechery by Ben Thompson

<https://stratechery.com/2024/the-gen-ai-bridge-to-the-future/>



Evolution of the Web

The Web Eats Everything in its Path

Graphics

Camera

Animation

Messaging

Location

Real-Time Messaging

Motion Input

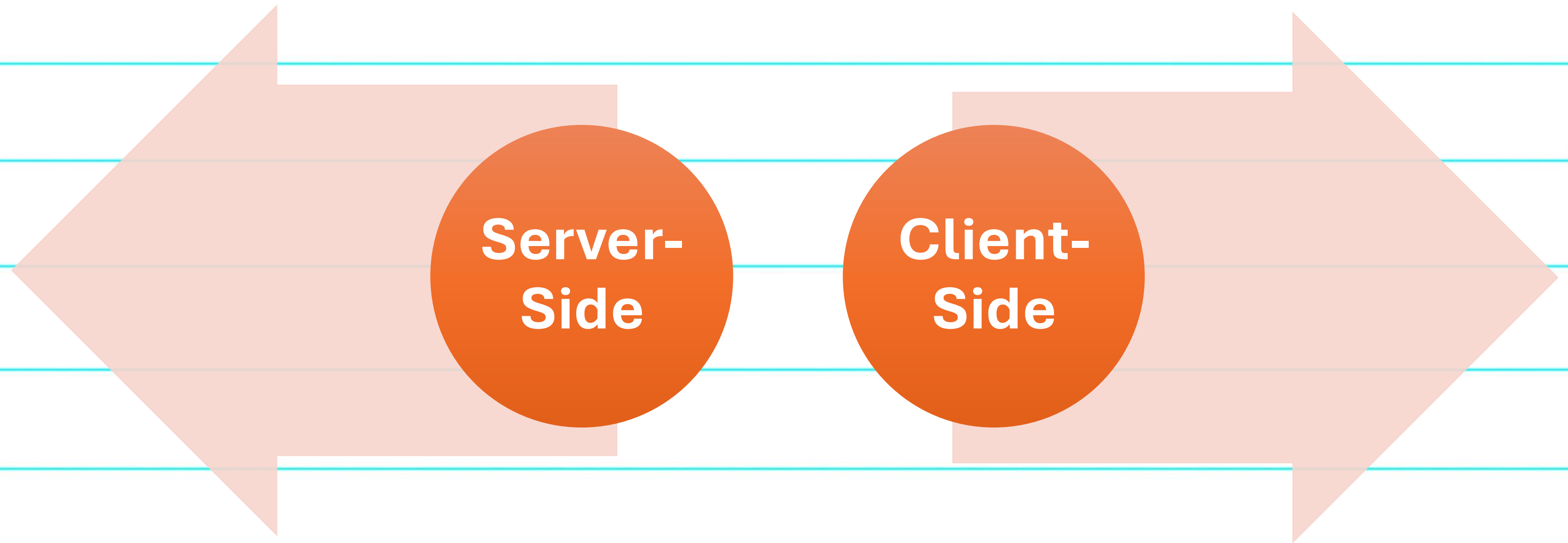
IOT/Wearables

Real-Time 3D

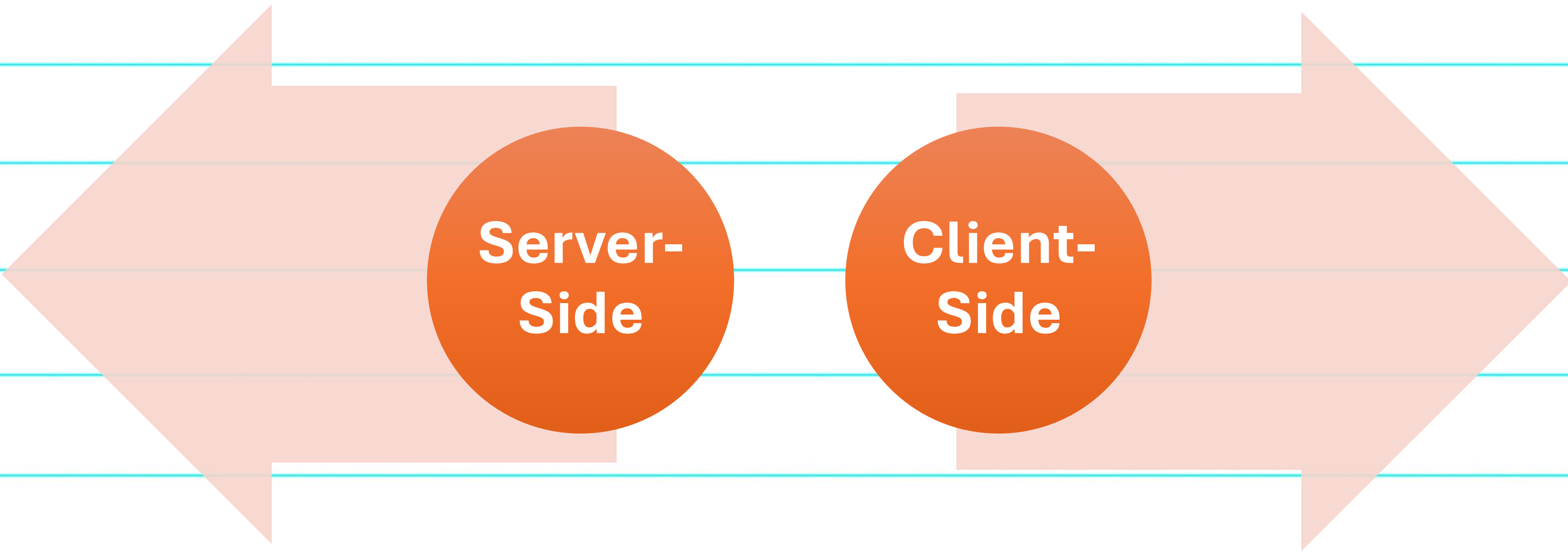
Robotics

And now AI

Choose Sides



JavaScript

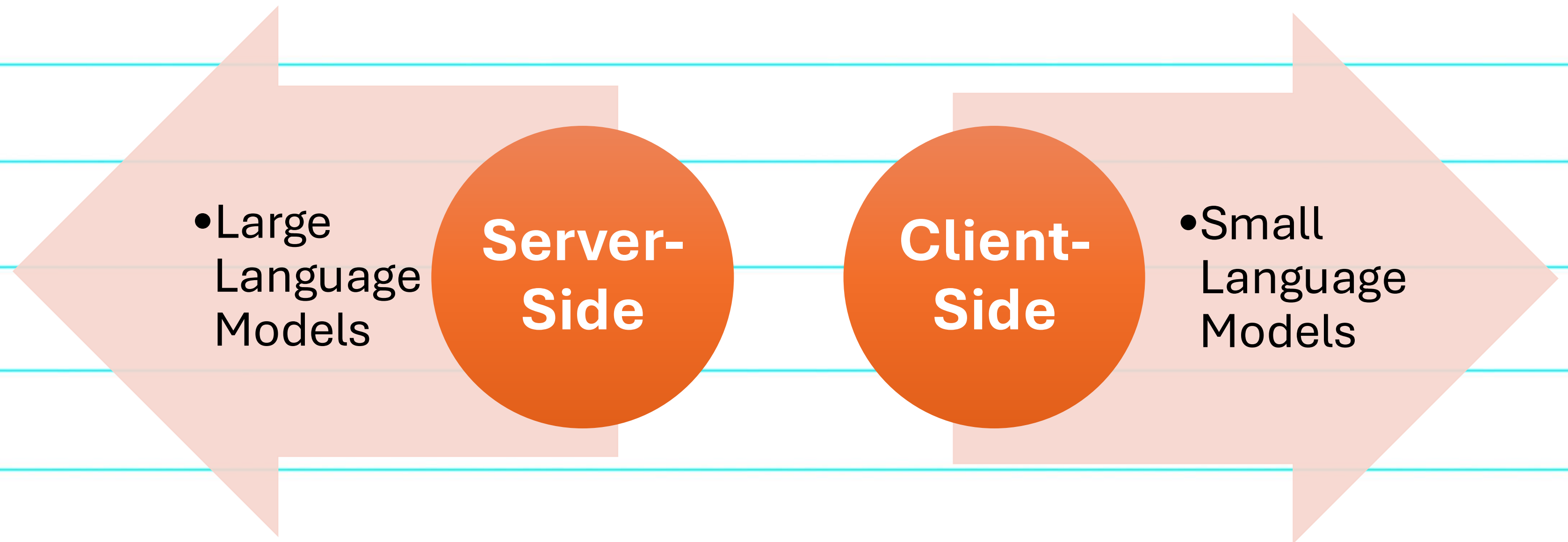


The diagram features two large, light-orange, semi-transparent arrows pointing in opposite directions, one to the left and one to the right, creating a sense of flow or interaction. In the center of each arrow is a solid orange circle. The left circle contains the text 'Server-Side' and the right circle contains the text 'Client-Side', both in white, bold, sans-serif font.

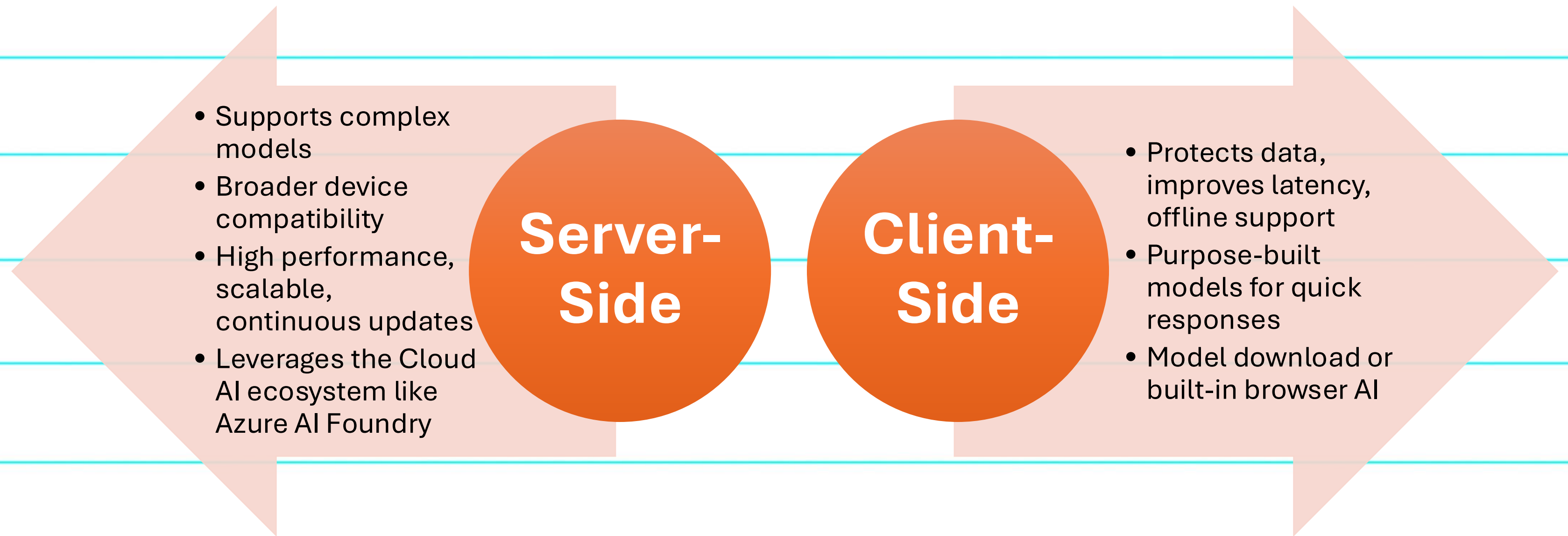
**Server-
Side**

**Client-
Side**

Foundation Models



Choose Sides



Choose Sides

Client-Side

- privacy, instant response, offline, lower cloud costs

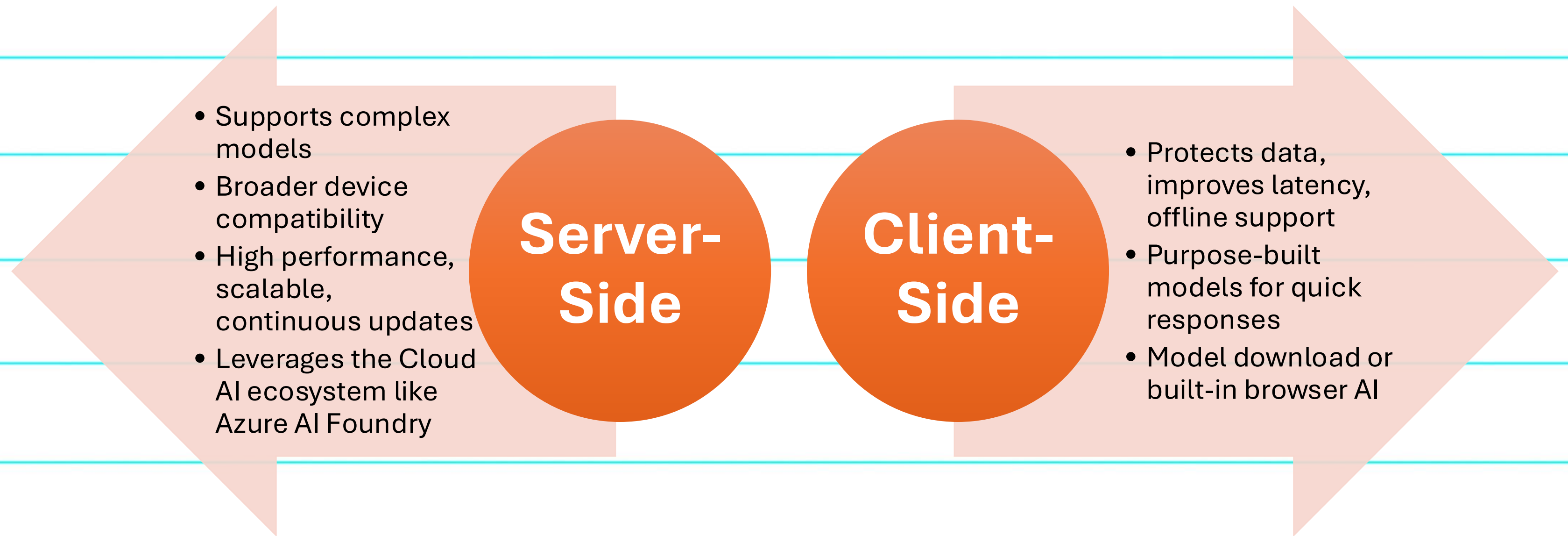
Server-Side

- heavier models, consistent perf, shared data, centralized updates

Hybrid

- small local models + server assist when needed

Choose Sides



Hybrid AI Approach

Client-Side

- Handles specific, approachable tasks

Server-Side

- For complex models & broader device compatibility

Graceful Fallback

- Ensures functionality on older or less powerful devices

Server-Side JavaScript

NodeJS

- TensorFlow.js
- Transformers.js
- LangChain.js
- LlamaIndex.ts
- Brain.js
- ML.js
- ONNX.js

**Server-
Side**

Why Server-Side AI?

Supports complex models

Broader device compatibility

High performance, scalable, continuous updates

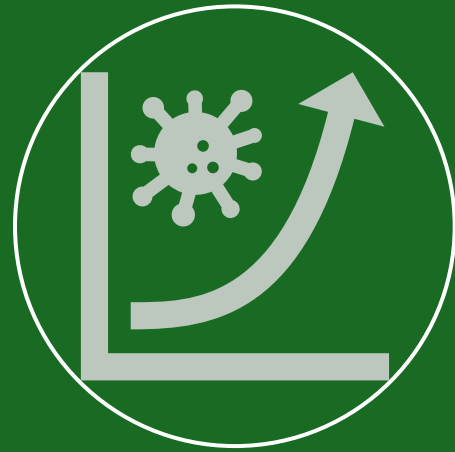
Centralized guardrails, audit logging, observability

Leverages the Cloud AI ecosystem like Azure AI Foundry

Benefits of Server-Side AI



**Data Center
Processing
Power**



Elastic scale



**Centralized
model
updates**



**Access to
shared, real-
time data**



**Predictable
experience
across
devices**

A Developer's Demo Prayer

Dear demo gods,

Give us strong Wi-Fi.

Keep pop-ups and updates away.

Let every API answer **200 OK**.

Hide our secret keys from the screen.

Keep the cloud calm and logs clean.

Guide our LLMs—no hallucinations quirk;

Tap **Run**, and boom—code will simply work.

In open-source we trust,

Amen.



LlamaIndex.TS

 Agents

Build agentic RAG applications

Truly powerful retrieval-augmented generation applications use agentic techniques, and LlamaIndex.TS makes it easy to build them.

```
import { VectorStoreIndex } from "llamaindex";
import { SimpleDirectoryReader } from "@llamaindex/readers/directory";
import { openai } from "@llamaindex/openai";
import { agent } from "@llamaindex/workflow";

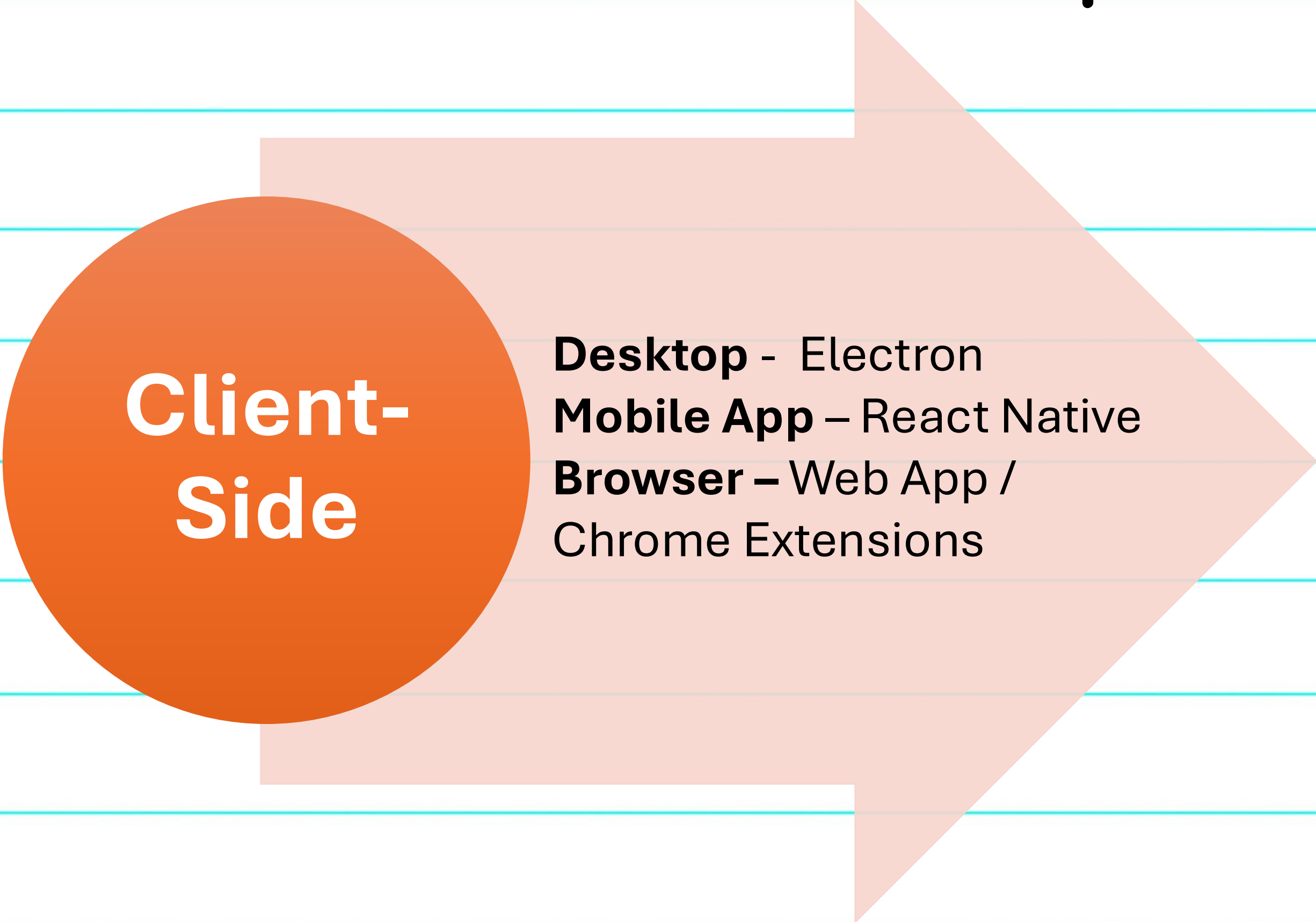
// load documents from current directory into an index
const reader = new SimpleDirectoryReader();
const documents = await reader.loadData(currentDir);
const index = await VectorStoreIndex.fromDocuments(documents);

const myAgent = agent({
  llm: openai({ model: "gpt-4o" }),
  tools: [index.queryTool()],
});

await myAgent.run('...');
```



Client-Side Javascript



The diagram features a large, light-orange arrow pointing to the right, which serves as a background for the content. On the left side of the arrow is a solid orange circle. The text 'Client-Side' is written in white inside this circle. To the right of the circle, within the arrow, are three lines of black text listing development environments: 'Desktop - Electron', 'Mobile App - React Native', and 'Browser - Web App / Chrome Extensions'.

Client-Side

Desktop - Electron

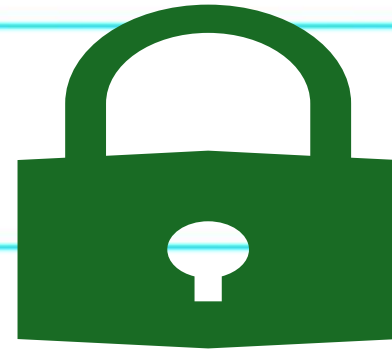
Mobile App - React Native

**Browser - Web App /
Chrome Extensions**

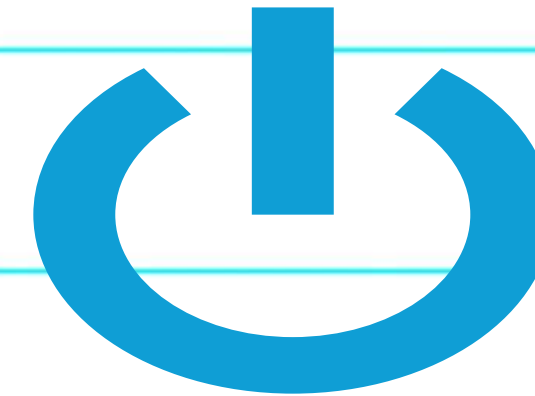
Why Client-Side AI?



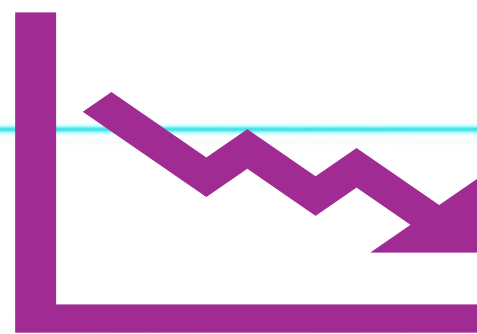
**Instant
response**



**Privacy by
default**



**Offline &
resilient**

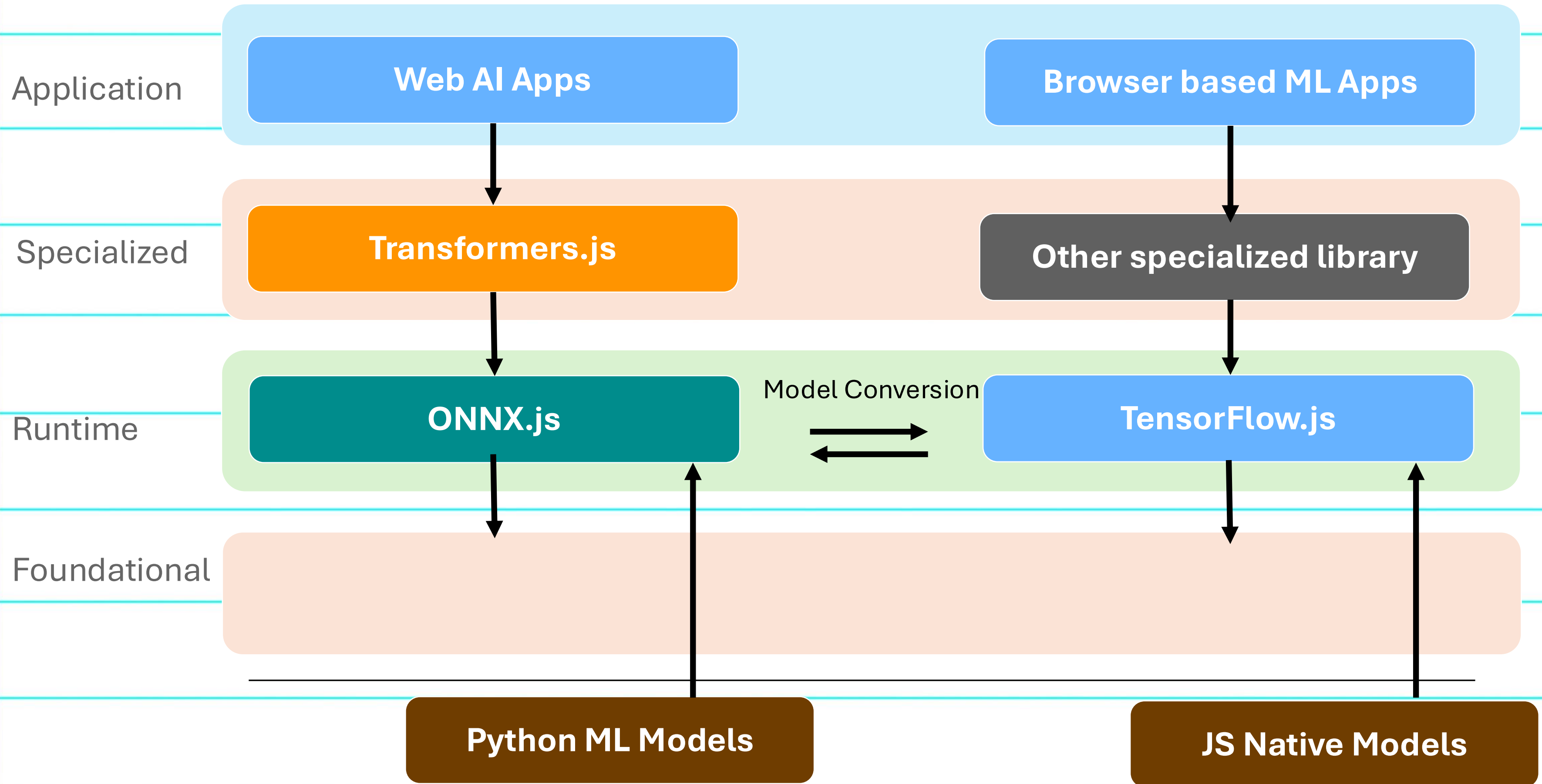


**Lower
cloud bills**



**Battery-aware
optimizations**

Tech Stack



Tensorflow.js

ML in JavaScript (browser + Node.js)

Great for custom models and
WebGL/WebGPU-accelerated ops

Dozens of ready models (pose, speech,
vision)



Tensorflow.js

JavaScript library for training and deploying machine learning models

Runs directly in web browser and Node.js

No need for server-side execution or specialized hardware



Tensorflow.js

Hardware Acceleration

Utilizes WebGL for improved performance

Faster computations directly in the browser



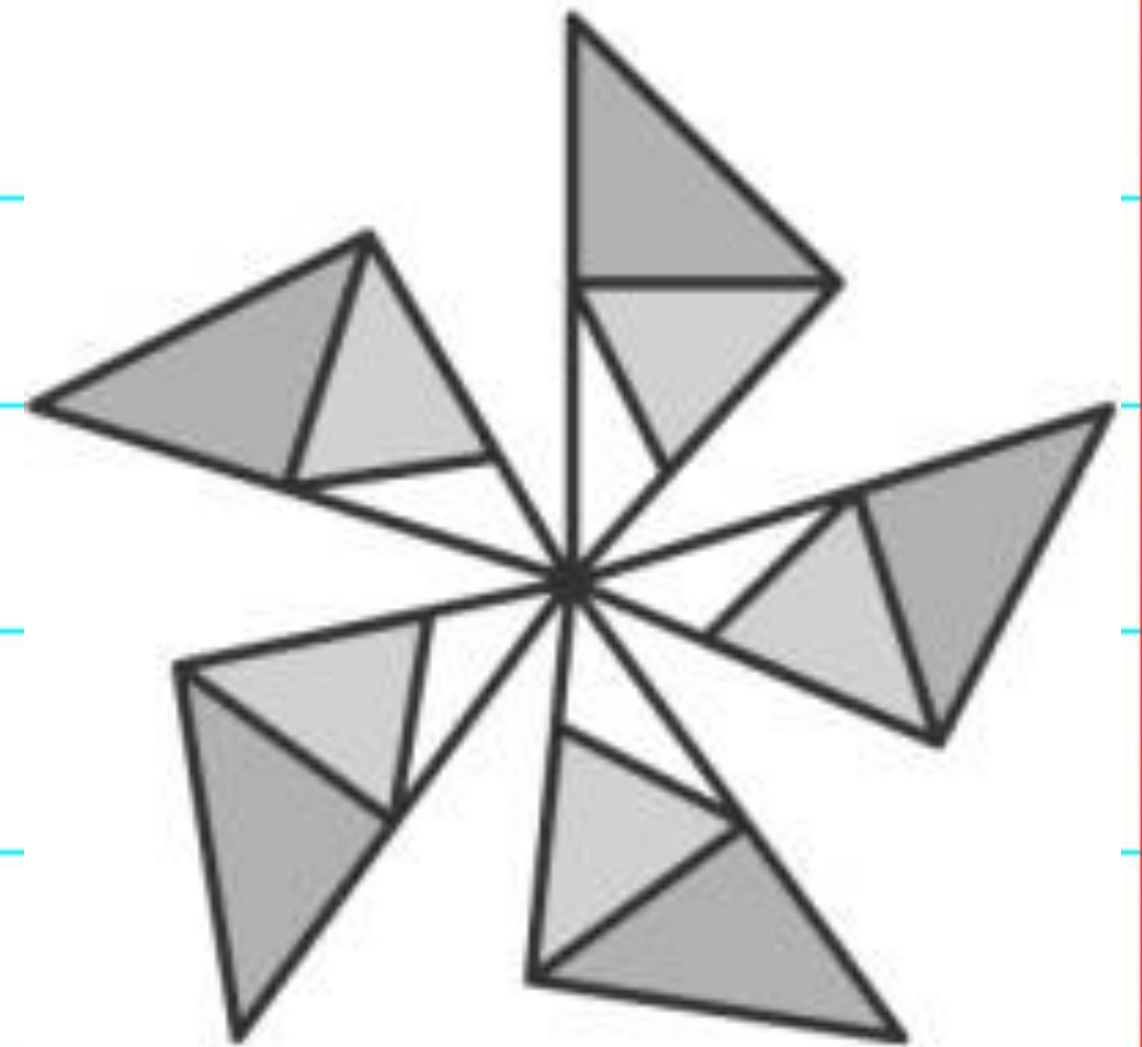
```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs"></script>
5     <script src="https://cdn.jsdelivr.net/npm/@tensorflow-models/toxicity"></script>
6     <script>
7       const threshold = 0.5
8       toxicity.load(threshold).then((model) => {
9         const sentences = ["You are a tiny, little baby poop", "My favorite color is blue.", "Shut up!"];
10
11         model.classify(sentences).then((predictions) => {
12           console.log(JSON.stringify(predictions, null, 2));
13         });
14       });
15     </script>
16   </head>
17   <body>
18     <h1>Check the console log!</h1>
19   </body>
20 </html>
```

ONNX

Open Neural Network Exchange (ONNX) is an open standard for ML models

Enables interoperability across various open source AI frameworks

Offers flexibility in adopting different AI frameworks

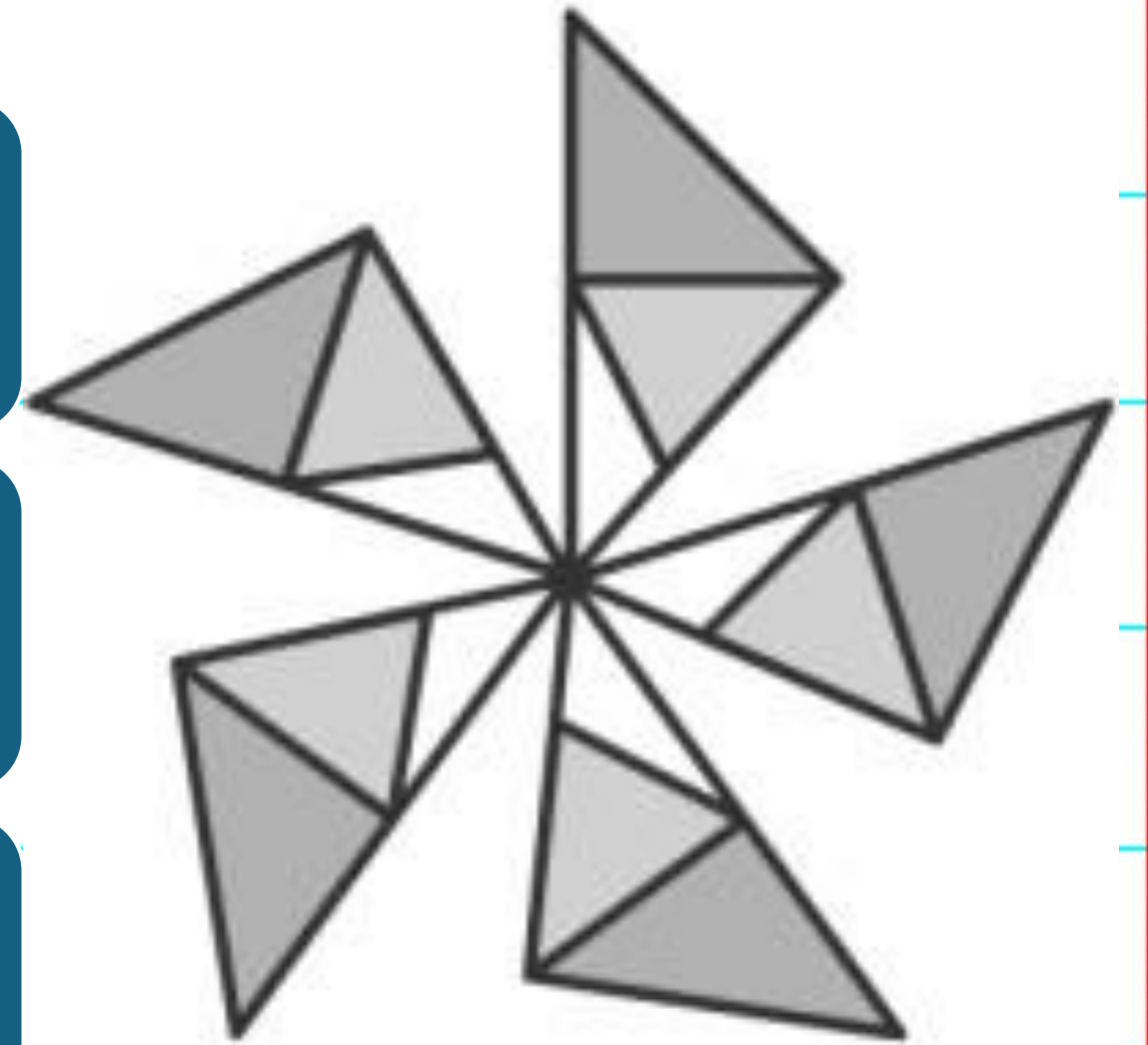


ONNX Runtime Web

Run ONNX models in the browser

Execution providers: WebAssembly (CPU),
WebGPU/WebGL (GPU), WebNN (where available)

Good path from Python training → JS inference

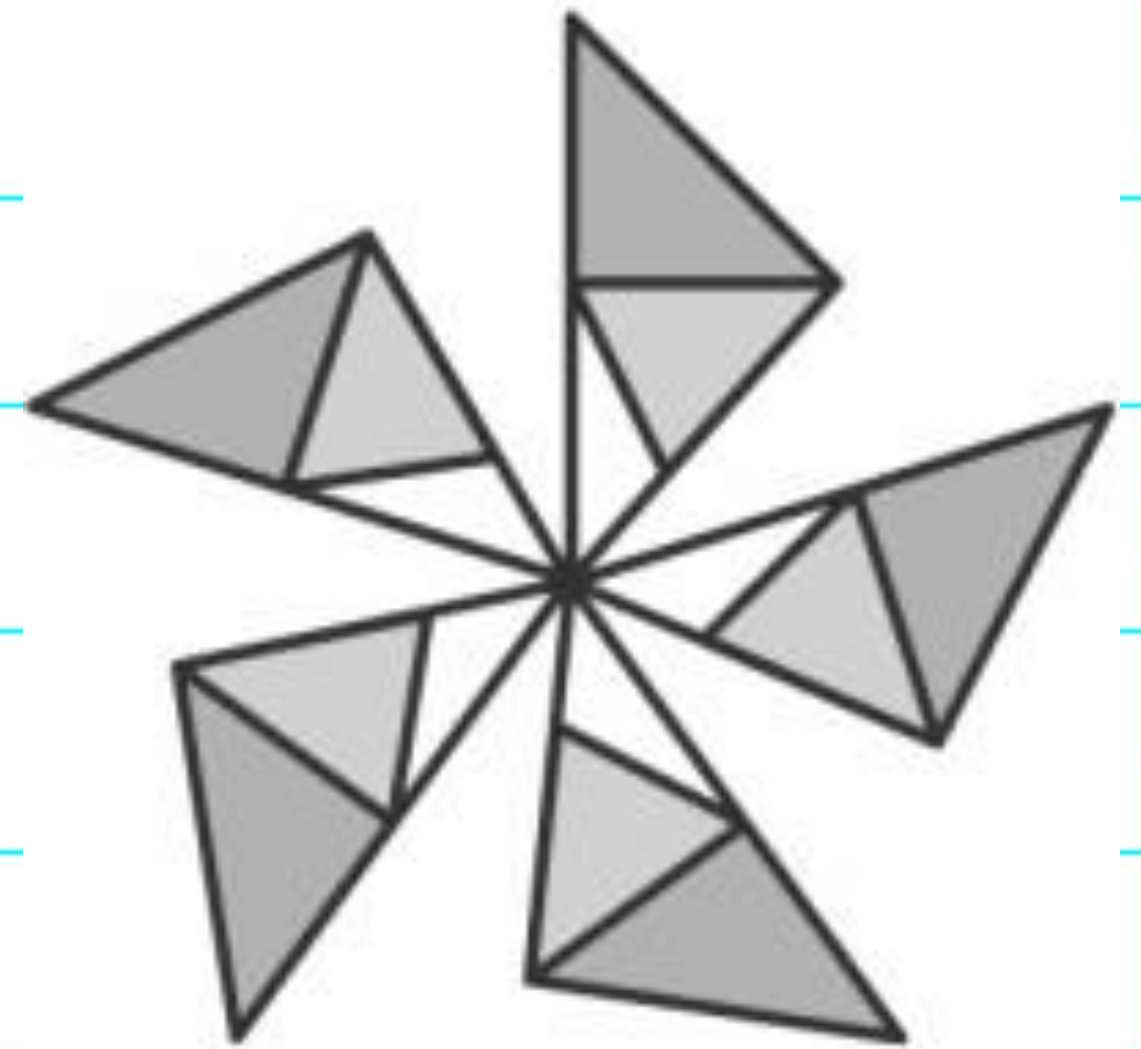


ONNX Runtime Web

A JavaScript library for running ONNX models

Supports both browser and Node.js environments

WebAssembly and WebGL



```
<script src="https://cdn.jsdelivr.net/npm/onnxruntime-web/dist/ort.min.js"></script>
```

```
<script>
```

```
    // use an async context to call onnxruntime functions.
```

```
    async function main() {
```

```
        try {
```

```
            // create a new session and load the specific model.
```

```
            //
```

```
            // the model in this example contains a single MatMul node
```

```
            // it has 2 inputs: 'a'(float32, 3x4) and 'b'(float32, 4x3)
```

```
            // it has 1 output: 'c'(float32, 3x3)
```

```
            const session = await ort.InferenceSession.create('./model.onnx');
```

```
            // prepare inputs. a tensor need its corresponding TypedArray as data
```

```
            const dataA = Float32Array.from([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]);
```

```
            const dataB = Float32Array.from([10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120]);
```

```
            const tensorA = new ort.Tensor('float32', dataA, [3, 4]);
```

```
            const tensorB = new ort.Tensor('float32', dataB, [4, 3]);
```

```
            // prepare feeds. use model input names as keys.
```

```
            const feeds = { a: tensorA, b: tensorB };
```

```
            // feed inputs and run
```

```
            const results = await session.run(feeds);
```

```
            // read from results
```

```
            const dataC = results.c.data;
```

```
            document.write(`data of result tensor 'c': ${dataC}`);
```

```
        } catch (e) {
```

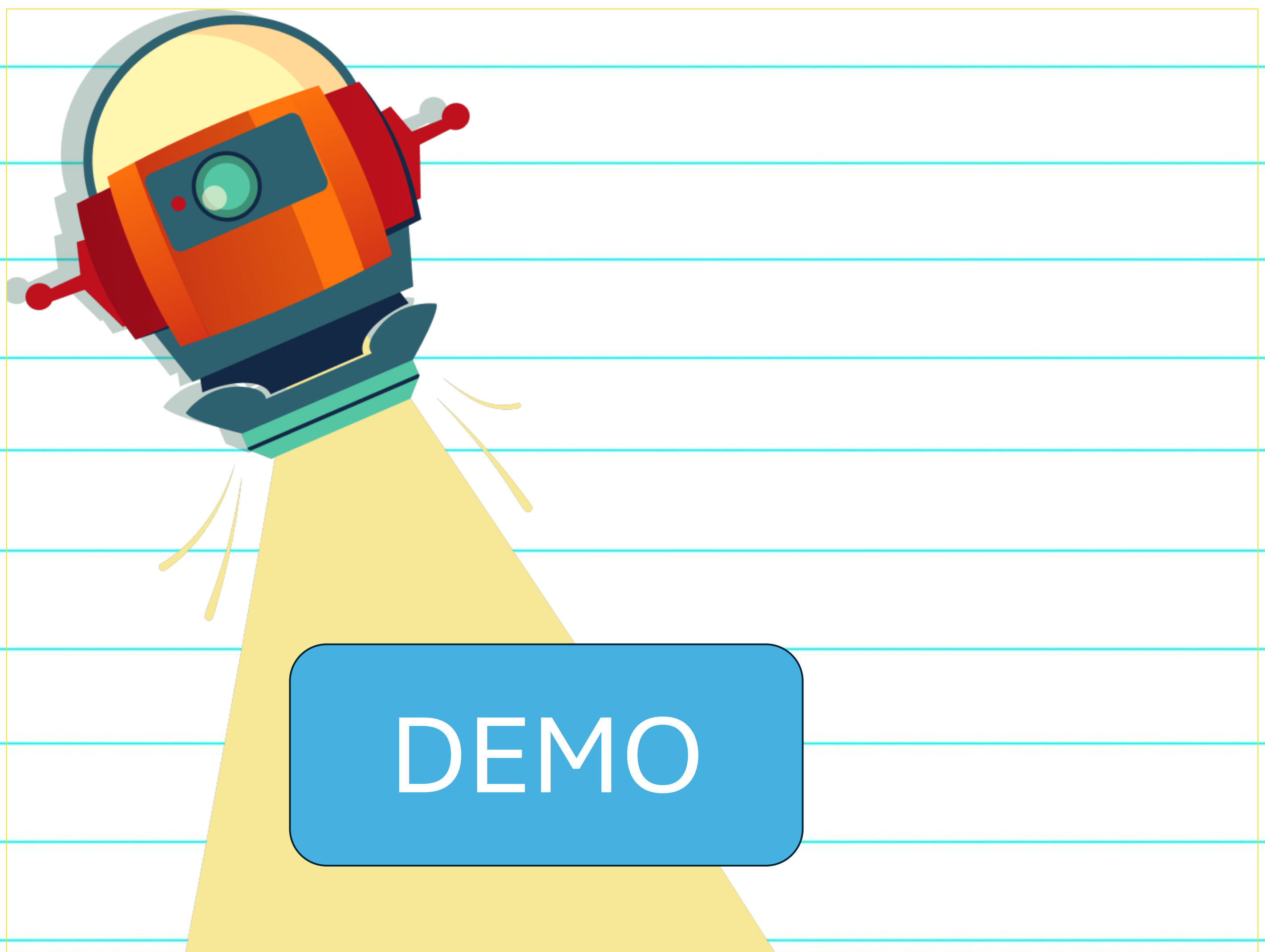
```
            document.write(`failed to inference ONNX model: ${e}.`);
```

```
        }
```

```
    }
```

```
    main();
```

```
</script>
```



Transformer.js

Run HF transformers in browser: text, vision, audio, multimodal

Works with pre-quantized models; streaming supported

Great demos/templates on Hugging Face



Transformer.js

- 📄 **Natural Language Processing:** text classification, named entity recognition, question answering, language modeling, summarization, translation, multiple choice, and text generation.
- 🖼️ **Computer Vision:** image classification, object detection, segmentation, and depth estimation.
- 🗣️ **Audio:** automatic speech recognition, audio classification, and text-to-speech.
- 🦑 **Multimodal:** embeddings, zero-shot audio classification, zero-shot image classification, and zero-shot object detection.



Transformers.js

```
import { pipeline } from '@huggingface/transformers'  
  
// Allocate a pipeline for sentiment-analysis  
const pipe = await pipeline('sentiment-analysis');  
  
const out = await pipe('I love transformers!');  
// [{ 'label': 'POSITIVE', 'score': 0.999817686 }]
```



Small Language Models

subset of language models

reduction in model size decreases complexity

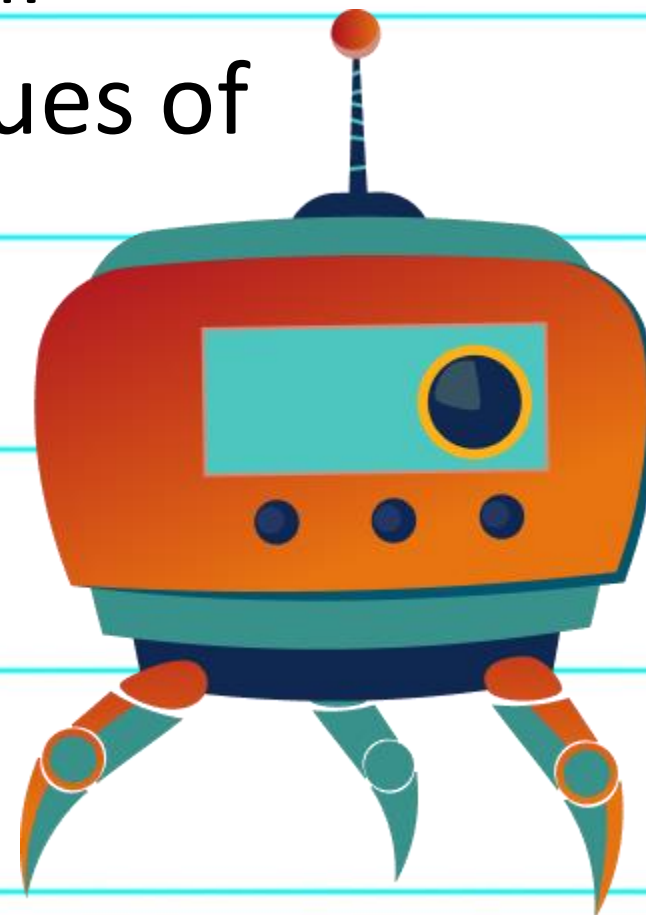
scaled-down variant of a large language model (LLM)

compact and efficient

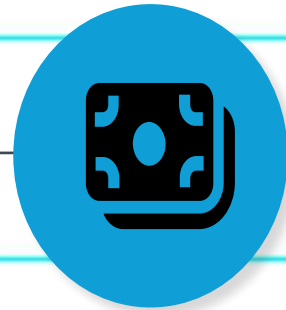
leveraging architectural principles and techniques of LLMs

significantly reduced computational footprint.

efficient in memory usage, computational requirements



Small Language Models



**Cost
Effectiveness**



**Deployment
Flexibility**



**Ultra-low
Latency**



**Easier to
Customize**

Phi Family Models

THE MICROSOFT PHI FAMILY: EDGE TO CLOUD SPECTRUM

LOCAL & EDGE (PRIVACY, LOW LATENCY)



MODELS: Phi-4 Mini (3.8B),
Phi-3.5 Mini, Phi-3 Vision

- Offline Capability
- Enhanced Privacy
- Enhanced Privacy
- Cost-Effective



CLOUD & ENTERPRISE (COMPLEX LOGIC, SCALE)



MODELS: Phi-4 Reasoning,
Phi-4 Multimodal, Phi-3.5 MoE

- Advanced Reasoning
- Advanced Reasoning
- Large-Scale Vision
- High Accuracy



UNIFIED INTELLIGENCE

Seamless AI from
Device to Data Center

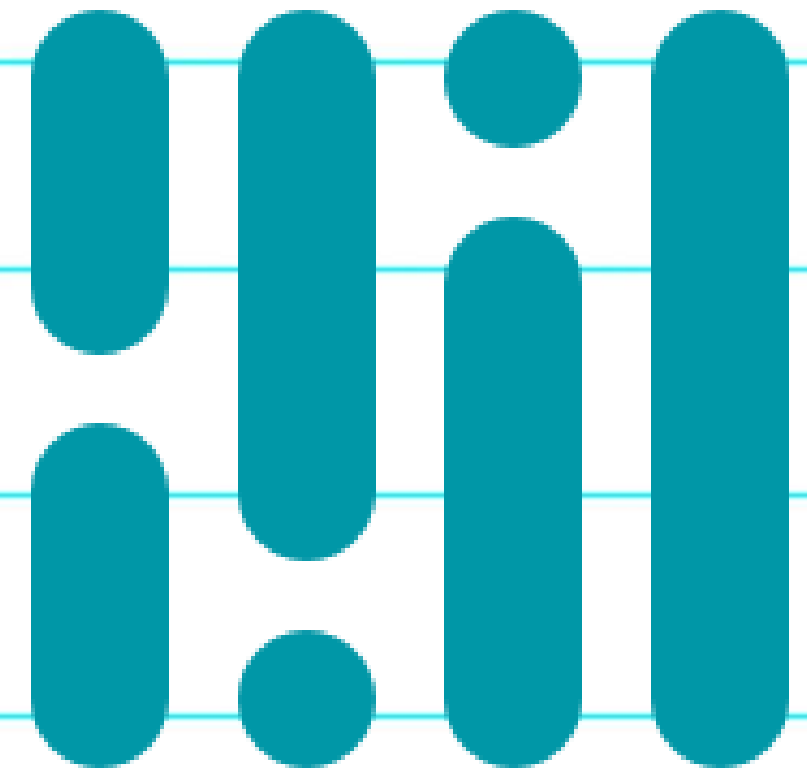


MediaPipe

Production-ready building blocks for vision/audio (Hands, Face, Pose, TTS, etc.)

JS/TS APIs with CDNs and Tasks SDK; runs on web and mobile

Great for overlays and AR-style interactions



MediaPipe

	C++	JS	Python	Android	iOS	<u>Coral</u>
<u>Face Detection</u>	✓	✓	✓	✓	✓	✓
Face Mesh	✓	✓	✓	✓	✓	
Iris Detection	✓			✓	✓	
Hands Detection	✓	✓	✓	✓	✓	
<u>Pose Detection</u>	✓	✓	✓	✓	✓	
Holistic	✓	✓	✓	✓	✓	
Selfie Segmentation	✓	✓	✓	✓	✓	
Hair Segmentation	✓			✓		
<u>Object Detection</u>	✓			✓	✓	✓
<u>Box Tracking</u>	✓			✓	✓	
Instant Motion Tracking				✓		
Objectron (3D)	✓	✓	✓	✓		
KNIFT				✓		
AutoFlip	✓					
MediaSequence	✓					
YouTube 8M	✓					



```
/**
 * This class uses the Singleton pattern to enable lazy-loading of the pipeline
 */
class TextGenerationPipeline {
    static model_id = "onnx-community/Phi-3.5-mini-instruct-onnx-web";

    static async getInstance(progress_callback = null) {
        this.tokenizer ??= AutoTokenizer.from_pretrained(this.model_id, {
            progress_callback,
        });

        this.model ??= AutoModelForCausalLM.from_pretrained(this.model_id, {
            // dtype: "q4",
            dtype: "q4f16",
            device: "webgpu",
            use_external_data_format: true,
            progress_callback,
        });

        return Promise.all([this.tokenizer, this.model]);
    }
}
```

```

    ✓ async function load() {
    ✓   self.postMessage({
      status: "loading",
      data: "Loading model...",
    });

    // Load the pipeline and save it for future use.
    ✓ const [tokenizer, model] = await TextGenerationPipeline.getInstance((x) => {
    ✓   // We also add a progress callback to the pipeline so that we can
      // track model loading.
      self.postMessage(x);
    });

    ✓ self.postMessage({
      status: "loading",
      data: "Compiling shaders and warming up model...",
    });

    // Run model with dummy input to compile shaders
    const inputs = tokenizer("a");
    await model.generate({ ...inputs, max_new_tokens: 1 });
    self.postMessage({ status: "ready" });
  }

```

Gemini Nano

two small models:

Nano-1

and the slightly more capable

Nano-2

which is meant to run offline.

unlocks new possibilities for AI agents - intelligent systems that can use memory, reasoning, and planning to complete tasks for you.

Built-in AI

- AI models built directly into web browsers
- Runs locally on devices, reducing dependency on servers
- Enhances privacy and performance

Built-in AI

Ease of Deployment

- Browser distributes & updates models
- No need to manage large downloads

Hardware Acceleration

- Optimized for GPUs, NPUs, CPUs

Local Processing

- Improves privacy & speeds up responses

Offline Usage

- AI features available without internet

Built-in AI

Privacy & Security

- Sensitive data stays on device

Snappy User Experience

- Near-instant results without server round trips

Cost & Scalability

- Offloads processing to users' devices

Enhanced Feature Access

- Preview premium features with minimal cost

Built-in AI

API	Explainer	Web	Extensions	Chrome Status	Intent
Translator API	MDN	 Chrome 138	 Chrome 138	View	Intent to Ship
Language Detector API	MDN	 Chrome 138	 Chrome 138	View	Intent to Ship
Summarizer API	MDN	 Chrome 138	 Chrome 138	View	Intent to Ship
Writer API	GitHub	 Origin trial	 Origin trial	View	Intent to Experiment
Rewriter API	GitHub	 Origin trial	 Origin trial	View	Intent to Experiment
Prompt API	GitHub	 Origin trial	 Chrome 138	View	Intent to Experiment
Proofreader API	GitHub	 Origin trial	 Origin trial	View	Intent to Experiment

Built-in AI

Not yet supported:

- Chrome for Android
- Chrome for iOS
- Chrome for ChromeOS

AI Chrome Extensions

Enhance Browsing

- Control and customize web content

Elevate User Experience

- Smarter bookmarks, new tab pages, history insights

Seamless Integration

- Side panels, action bars, context menus

Explain in Generations



```
// Set up the context menu item when the extension is installed or updated
✓ chrome.runtime.onInstalled.addListener(() => {
✓   chrome.contextMenus.create({
      id: "summarize-text",
      title: "Explain in Generations",
      contexts: ["selection"] // Only show menu on text selection
    })
  })

✓ // Helper function to get the current summarization options
  // These options configure how the AI explains the selected text
✓ const getOptions = () => ({
    sharedContext: `${currentLevel.context}. ${currentLevel.description}`,
    type: "tldr",
    format: "plain-text",
    length: "medium"
  })
```

```
=
{
  level: 5,
  name: "Millennial",
  context:
    "Summarize this for a Millennial (born 1980–1994). Make it relatable and engaging like a BuzzFeed article. Use analogies to: AIM away messages, burning CDs, iPod playlists, MySpace Top 8, early Facebook, flip phones, Harry Potter houses, Friends episodes, Pokémon evolutions, and Netflix DVD mailers. Explain concepts like 'machine learning is like how Spotify learned your music taste over time' or 'blockchain is like a Google Doc everyone can see but no one can secretly edit.' Use phrases like 'okay so basically,' 'here's the thing,' and 'think of it like.' Reference the analog-to-digital childhood transition. Keep it authentic, slightly nostalgic, and easy to share.",
  description:
    "BuzzFeed-style with AIM and iPod analogies, Harry Potter references, Spotify-era relatable explanations"
},
{
  level: 6,
  name: "Generation Z",
  context:
    "Summarize this for Generation Z (born 1995–2012). Keep it short, visual, and real. Make analogies to: Instagram filters, TikTok algorithms, Discord servers, Minecraft worlds, YouTube recommendations, Spotify Wrapped, iPhone shortcuts, Among Us tasks, and Netflix autoplay. Explain concepts like 'an API is like how apps talk to each other, like when Spotify shares to your Instagram story' or 'encryption is like sending a Snap that only the right person can open.' Use phrases like 'basically,' 'think of it like,' and 'the tldr is.' Break into scannable chunks. Be authentic, skip the corporate speak, and acknowledge when something is actually complicated.",
  description:
    "Short scannable format with TikTok and Discord analogies, Instagram-filter references, Spotify Wrapped-style explanations"
},
{
  level: 7,
  name: "Generation Alpha",
  context:
    "Summarize this for Generation Alpha (born 2013–2025). Make it fun, interactive, and visual. Use analogies to: Roblox experiences, iPad apps, YouTube Kids, voice assistants like Alexa, smart home devices, Minecraft redstone circuits, tablet games, and educational apps like Khan Academy Kids. Explain concepts like 'the internet is like a giant Roblox world where everyone builds and shares things' or 'a password is like your secret code to unlock your favorite game.' Use emojis strategically 🎮✨, break into small chunks, and make it feel like a fun discovery. Use phrases like 'imagine if,' 'it's kind of like,' and 'here's the cool part.' Keep it positive, safe, and age-appropriate.",
  description:
    "Fun interactive style with Roblox and iPad analogies 🎮, YouTube Kids references, game-world discovery explanations ✨"
```

```
const availability = await Summarizer.availability();  
let summarizer  
  
if (availability === 'unavailable') {  
  chrome.runtime.sendMessage({  
    type: "ERROR",  
    error: "The Summarizer API isn't usable"  
  })  
  return  
}  
  
if (availability === 'available') {  
  chrome.runtime.sendMessage({  
    type: "STREAM_RESPONSE",  
    isFirst: true,  
    level: currentLevel.level  
  })  
  
  // Initialize summarizer  
  // @ts-expect-error new chrome feature  
  summarizer = await Summarizer.create(getOptions());  
  await summarizer.ready  
  
  // Process text and stream results  
  const stream = await summarizer.summarize(info.selectionText, {  
    context: `article from ${new URL(tab.url!).origin}`  
  })  
  
  for await (const chunk of stream) {  
    await chrome.runtime.sendMessage({  
      type: "STREAM_RESPONSE",  
      chunk,  
      level: currentLevel.level  
    })  
  }  
}
```

Session Lifecycle

```
// Create session with options
let session = await self.Summarizer.create({
  type: 'key-points',
  format: 'markdown',
  length: 'short'
});

// Use session
let summary = await session.summarize(text);

// Always cleanup
session.destroy();
```



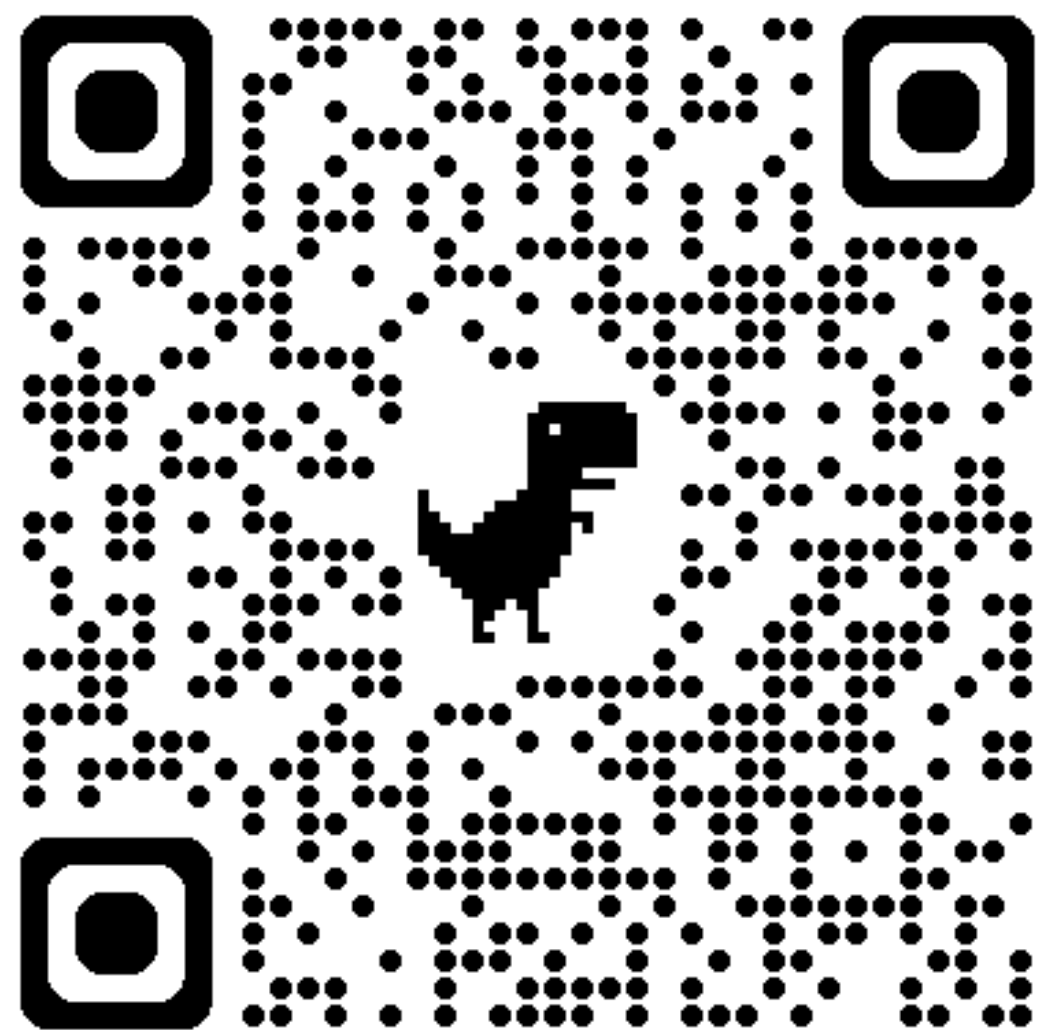
Why AI in JS

- Python remains the powerhouse for research & model training.
- Client-Side and Real-time, user-facing AI experiences run on JavaScript.
- JavaScript bridges complex models to seamless UX across web & mobile.
- Leverage existing JS skills to build AI-driven apps.
- Tools are ready.

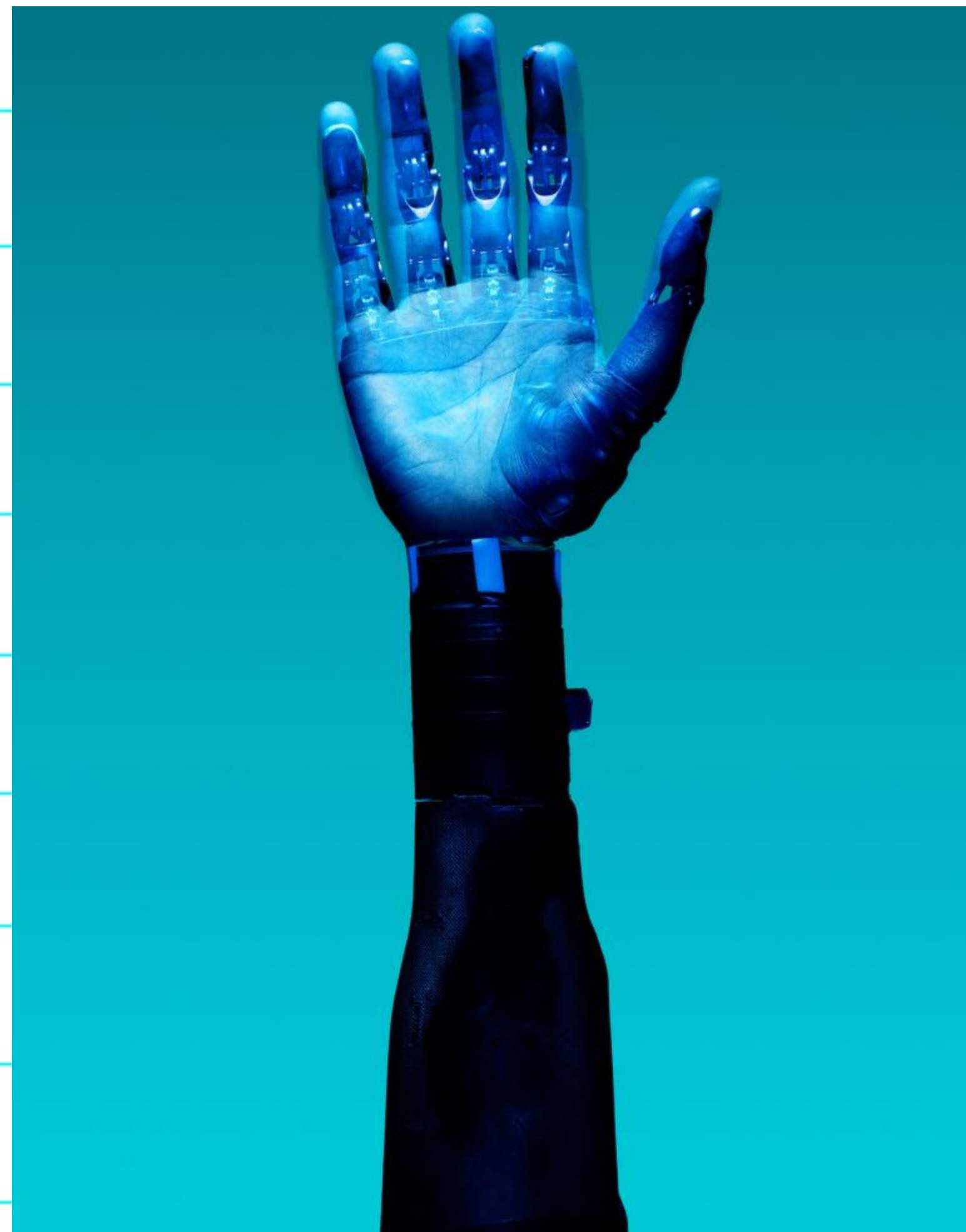
AI



JS



Questions?





Microsoft®
Most Valuable
Professional



Award Categories

AI, Windows Development,
Internet of Things, Mixed Reality

Ron Dagdag

R&D Engineering Manager at 7-Eleven

10th year Microsoft MVP awardee

www.dagdag.net

[@rondagdag](https://twitter.com/rondagdag)

Linked In
www.linkedin.com/in/rondagdag/

Thanks for geeking out about AI in JavaScript

